



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 5

Team No.: 19

Project Title: LinuxServiceProcessSupervisor

Team Members

Roll Number	Student Name	Signature
2520030380	Yamini Katta	
2520030364	Lekhana L	
2520030613	Anikha	

1. Abstract

LinuxServiceProcessSupervisor is a Linux-based Operating Systems project that implements a lightweight service supervisor, similar in spirit to tools such as `systemd` and `supervisord`, which monitors, manages, and automatically restarts application processes running on a Linux system. In real-world environments, long-running services can crash, hang, or terminate unexpectedly, and without an automated supervision mechanism, such failures can go unnoticed and disrupt system functionality. This project addresses that problem by building a custom process supervisor that launches child processes as managed “services,” continuously monitors their status, and takes corrective action when a service fails.

The project focuses on key Operating Systems and system programming concepts such as **process creation, process identification, process monitoring, inter-process communication, signal handling, process synchronization, and process control**. System calls and functions such as `fork()`, `execve()`, `waitpid()`, `kill()`, `signal()/sigaction()`, and `pause()` are used to launch, monitor, and control the lifecycle of supervised services.

The proposed solution consists of a Linux-based supervisor application written in **C/C++**, where a parent supervisor process reads a simple configuration listing the services to be run, forks a child process for each service, and continuously tracks its state. If a service process terminates unexpectedly, the supervisor automatically restarts it and logs the event. The expected outcome is a functional service-supervision utility that provides practical understanding of how Linux process supervisors such as `systemd` operate internally, while strengthening system-level programming skills in process management, signal handling, and fault-tolerant process control.

2. Problem Statement

In a Linux environment, multiple background services and long-running processes must remain continuously available. However, processes can fail due to crashes, resource exhaustion, or unexpected termination. Without an

automated supervision mechanism, a failed service may remain down until it is manually detected and restarted by an administrator, leading to downtime and reduced system reliability.

This project addresses the problem of **automated process supervision and fault recovery** using Linux system calls. It demonstrates how a supervisor process can launch, monitor, and restart dependent child processes without relying on external, pre-built tools. The project is relevant to Operating Systems and Systems Programming because it involves practical concepts such as process creation, process identification, inter-process communication, signal handling, process synchronization, and process termination.

The proposed system uses Linux system calls to allow a supervisor (parent) process to start, monitor, restart, and gracefully stop one or more managed service (child) processes. This provides a practical understanding of how Linux-based init systems and service managers maintain system reliability.

The project aims to demonstrate that a custom-built, signal-aware process supervisor can provide a simple yet effective mechanism for keeping services running reliably in a Linux environment.

3. Objectives

1. To understand the concept of process supervision and fault-tolerant process management in Linux.
2. To create and manage multiple service processes using the `fork()` and `execve()` system calls.
3. To implement continuous process monitoring using `waitpid()` and `SIGCHLD` handling.
4. To implement supervisor commands for service control such as start, stop, restart, and status check.
5. To automatically detect service failure and restart the affected process, with restart attempts logged.
6. To demonstrate inter-process communication and synchronization between the supervisor and its managed services using signals.

4. Proposed Methodology

The project will be designed and implemented as a Linux-based C/C++ supervisor application that launches, monitors, and manages the lifecycle of one or more service processes.

1. Configuration Loading: The supervisor reads a simple configuration (a list of service names and the commands used to launch them).
2. Process Creation: The supervisor uses `fork()` and `execve()` to launch each configured service as a child process.
3. Process Identification: Each service process is tracked using its Process ID (PID), and the supervisor maintains a table mapping service names to PIDs and states.
4. Signal Handling: A `SIGCHLD` handler is registered using `sigaction()` so the supervisor is notified asynchronously whenever a managed service terminates.
5. Process Monitoring and Recovery: When a service exits unexpectedly, the supervisor detects this through `waitpid()` and automatically restarts the service, logging the restart with a timestamp and reason.

6. Process Control: The supervisor exposes control operations that send signals via `kill()` to managed services, for example `SIGSTOP/SIGCONT` to pause/resume a service and `SIGTERM` to stop it gracefully.

7. Synchronization: The supervisor uses `waitpid()` (non-blocking, with `WNOHANG`) in its monitoring loop to reap terminated children and update service status without blocking other supervisor operations.

8. Execution Flow: The overall flow will be: Load Configuration → Fork & Launch Services → Register Signal Handlers → Monitor Services → Detect Failure/Command → Restart or Control Service → Log Status → Graceful Shutdown of All Services.

9. Testing: The supervisor will be executed in a Linux terminal and tested by deliberately crashing, stopping, and terminating sample services to verify that monitoring, automatic restart, and control commands function correctly.

The methodology provides a practical demonstration of process management, automated fault recovery, signal-based control, and Linux system calls.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation – <code>fork()</code> , <code>execve()</code>	Launches each configured service as an independently running child process.
Process Identification – <code>getpid()</code> , <code>getppid()</code>	Tracks the PID and PPID of the supervisor and each managed service.
Signal Handling – <code>signal()</code> / <code>sigaction()</code>	Registers a <code>SIGCHLD</code> handler so the supervisor is notified when a service terminates.
Process Monitoring – <code>waitpid()</code> (<code>WNOHANG</code>)	Reaps terminated child processes and detects failures without blocking the supervisor.
Signal Sending – <code>kill()</code>	Sends control signals from the supervisor to managed services (stop, pause, resume).
Process Control – <code>SIGSTOP</code> , <code>SIGCONT</code> , <code>SIGTERM</code> , <code>SIGKILL</code>	Implements pause, resume, graceful termination, and forced termination of services.
Automatic Restart Logic	Restarts a service automatically when it exits unexpectedly, with logging of each event.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030380	Yamini Katta	Process Creation & Configuration Module – Designed and implemented the service-launch logic using <code>fork()</code> and <code>execve()</code> ; built the configuration reader that lists services and their launch commands; maintained the PID/state tracking table for all supervised services.
2520030364	Lekhana L	Monitoring & Auto-Restart Module – Implemented the <code>SIGCHLD</code> signal handler and <code>sigaction()</code> setup; developed the <code>waitpid()</code> (<code>WNOHANG</code>) monitoring loop that detects service crashes/exits and triggers automatic restarts; implemented event logging with timestamps.
2520030613	Anikha	Process Control, Testing & Documentation – Implemented supervisor control commands (start, stop, pause/resume, status) using <code>kill()</code> with <code>SIGSTOP/SIGCONT/SIGTERM</code> ; carried out integration testing by simulating crashes and manual stop/restart scenarios; prepared documentation and GitHub repository.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing, compiling, and executing the supervisor program.
C / C++	Used to implement process creation, monitoring, signal handling, and service control logic.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Provides APIs and functions for process management, signals, and synchronization.
Linux Terminal / Shell	Used to compile, execute, test, and observe supervised services and their states.

8. Expected Outcome

The project is expected to produce a functional Linux-based service process supervisor that demonstrates how a set of managed processes can be launched, monitored, and automatically recovered from failure.

1. The supervisor will successfully launch one or more configured service processes using `fork()/execve()`.
2. Each supervised service will be tracked by PID, with its current state (running, stopped, restarting) visible to the supervisor.
3. The supervisor will detect service crashes or unexpected exits and automatically restart them.
4. The supervisor will be able to send control signals to services to pause, resume, or stop them on demand.
5. Signal handlers will demonstrate how the supervisor responds asynchronously to service termination events in real time.
6. The program will properly synchronize with and reap child processes using `waitpid()`, avoiding zombie processes.
7. The terminal/log output will clearly show service startup, monitoring activity, failure detection, automatic restarts, and graceful shutdown.
8. Overall, the project will provide practical understanding of Linux process supervision, fault-tolerant process control, signal-based IPC, and system programming, while demonstrating how tools like `systemd/supervisord` operate at a fundamental level.

9. GitHub Repository

GitHub Repository Link:

Repository Created: ☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved: ☐ Yes ☐ No

Date:

Faculty Signature: